

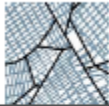
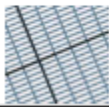
Team Cityscape Report

Members: Luke Knight (lck592)

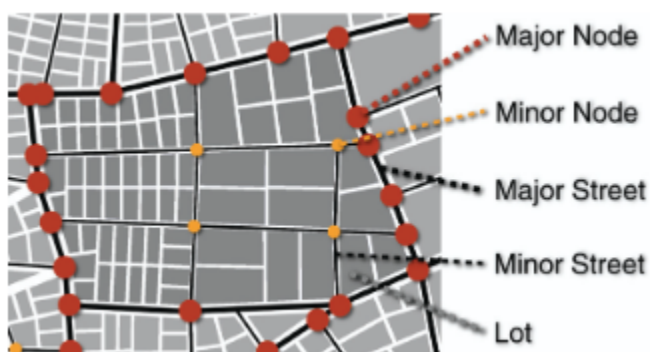
Repository: <https://github.com/lukektx/CityGeneration>

Background

My project focuses on creating procedural city layouts and simulating traffic agents traveling along the roads. This was initially inspired by Parish and Müller's paper using L-systems as well as LocalConstraint and GlobalGoal systems to govern generation. I began my project implementing a method similar to this. The important thing to know is that LocalConstraints governs strict placement rules such as avoiding water, minimum connected angles, snapping nearby vertices together, and similar. GlobalGoals focused on road generation patterns

Pattern name	Pattern	Example
Basic	No superimposed pattern.	
New York	Rectangular Raster	
Paris	Radial to center	

My current version, which was adapted to follow more closely to the procedures used by Basil et al, still utilizes the LocalConstraints to find valid road segments, but at this time doesn't support different road patterns. Some important terminology to know for future descriptions relates to different subdivisions of land and road types.



As shown, there are wider major streets and thinner and slower minor streets. Nodes along a major street in the road graph are referred to as major nodes, and ones only

connected to minor streets are minor nodes. The larger polygon enclosed by streets is called a “quarter” which in the image above is divided further into the gray polygons called “lots.” In my simulation, the generation stops at quarters, and they aren’t divided into lots, though to implement more in depth simulation, I hope to add this in the future.

Main Features

Road Generation

Street Expansion

The first main aspect of the roadmap generation is expanding streets. The roads are modelled as a graph with vertices at intersections, and roads as edges connecting them. The edges are modelled using two “half-edges,” as described by Basil et al, meaning a corresponding pair of directional edges connecting nodes in both directions, which end up being useful for cycle/quarter detection later (2009). The general rules for expanding roads are quite simple. The user can specify growth centers (in my version this is changeable in the Unity Editor, but not in the executable GUI) where a major road will be seeded. From here, the algorithm will select a random node with open sides to expand. A node is “finished” expanding once it has a valence (degree) of 4 or more because this means all four cardinal directions are likely filled. When choosing a node to expand, the algorithm (defined in Assets/Scripts/Core/Generation/StreetExpander.cs) will choose a node weighted by distance from the growth center. This is also influenced by a “target branch ratio” which describes the amount of straight intersections to four-way intersections and this ratio is maintained throughout generation.

```
private RoadNode? SampleNextMajorNode()
{
    // Valence 0 always has highest priority (new unconnected nodes)
    RoadNode? sample;
    if (TrySampleMajorValence(0, out sample)) return sample;

    // Valence 3 has next priority (expand to full junction)
    if (TrySampleMajorValence(3, out sample)) return sample;
    float currentRatio = graph.MajorValenceRatio;

    // Check if we need more valence 2 or 4 based on parameters
    bool needMoreBranches = currentRatio < parameters.TargetBranchRatio;

    if (needMoreBranches)
    {
        if (TrySampleMajorValence(2, out sample)) return sample;
        if (TrySampleMajorValence(1, out sample)) return sample;
    }
    else
    {
        if (TrySampleMajorValence(1, out sample)) return sample;
        if (TrySampleMajorValence(2, out sample)) return sample;
    }

    return null;
}
```

The actual algorithm for determining initial street placement is simple. If the node has a valence of 1, the next street goes straight forward, then at valence 2 (a straight line), it randomly chooses to expand left or right, and at valence 3 it chooses the cardinal direction determined to be unfilled based on angle deltas from the incoming segment. A configurable amount of random noise is added to the angle to produce some deviation from a strict grid pattern.

```
private float? GetExpansionAngle(RoadNode node)
{
    float? angle = node.Valence switch
    {
        0 => node.BaseAngle,
        1 => node.BaseAngle + 180f,
        2 => Random.value > 0.5f ? node.BaseAngle + 90f : node.BaseAngle - 90f,
        3 => ValenceThreeAngle(node),
        _ => null
    };
    return angle;
}
```

After a road segment candidate has been created from the starting node and angle, this is then evaluated by LocalConstraints. It will check for endpoints in water, snapping to nearby vertices, minimum angle difference to avoid nearly parallel roads and minimum road length to prevent tiny segments. The modified road candidate is then returned and created if a suitable candidate was found

```
private bool CommitSegment(RoadSegment segment)
{
    // Test LocalConstraints to apply global rules
    var result = LocalConstraints.Apply(segment, graph, parameters);
    if (result == ConstraintResult.Failed) return false;

    RoadNode toNode = segment.EndNode ?? graph.AddNode(segment.End, segment.Type);

    // Add edge to the graph
    var edge = graph.AddEdge(segment.StartNode!, toNode, segment.Type);

    // Check for new quarters when adding major roads
    if (segment.Type == RoadType.Major)
    {
        CheckForNewQuarters(edge);
    }

    return true;
}
```

Quarter Filling

The next part of the road generation is seen above as well, checking for quarter formation after adding major roads. As a reminder, a quarter refers to a polygon enclosed by major road segments, and our goal is to fill this with a mostly uniform grid of minor roads. This was probably one of the most challenging parts of my project and one where I ran into a significant amount of bugs. The system I settled on was that each incoming half-edge maintains a reference to the next outgoing half-edge CCW from it. This is updated whenever a node gets a new half-edge so it stays valid as the graph is updated. This makes it easy to find enclosed polygons to detect quarter formation.

```
public static List<Quarter> FindQuartersForEdge(
    RoadEdge newEdge,
    HashSet<Quarter> knownQuarters,
    float minArea = 100f
)
{
    if (newEdge.Type != RoadType.Major) return new List<Quarter>();

    var newQuarters = new List<Quarter>();

    // Each half-edge traces a different face
    foreach (var startHalf in new[] { newEdge.HalfA, newEdge.HalfB })
    {
        var face = TraceFace(startHalf);
        if (face == null || face.Count < 3) continue;

        var quarter = new Quarter(face);

        // Check quarter winding
        if (quarter.IsClockwise) continue;

        // Filter degenerate quarters
        if (quarter.Area < minArea) continue;

        if (knownQuarters.Add(quarter))
        {
            newQuarters.Add(quarter);
        }
    }
}
```

After finding new quarters that are formed the next step is to fill them with minor roads. The method I use for this, is to seed with one minor road, then expand from here. I

try expanding from each boundary node on the perimeter of the quarter, and if none work, then try expanding from the centroid of the polygon.

```
private void FillQuarter(Quarter quarter)
{
    // HashSet<RoadNode> interiorNodes = graph.Nodes.Where(node => quarter.IsPointInside(node.Position)).ToHashSet();
    // interiorNodes.UnionWith(quarter.Nodes.ToHashSet());
    HashSet<RoadNode> boundaryNodes = quarter.Nodes.ToHashSet();

    foreach (RoadNode node in boundaryNodes)
    {
        foreach (float angle in quarter.GetGridAngles())
        {
            // Stop after seeding one segment
            if (ExpandQuarterNode(node, angle, quarter)) return;
        }
    }

    RoadNode seedNode = graph.AddNode(quarter.Centroid, RoadType.Minor);
    seedNode.InitialAngle = Mathf.Atan2(quarter.MainAxis.y, quarter.MainAxis.x) * Mathf.Rad2Deg;
}
```

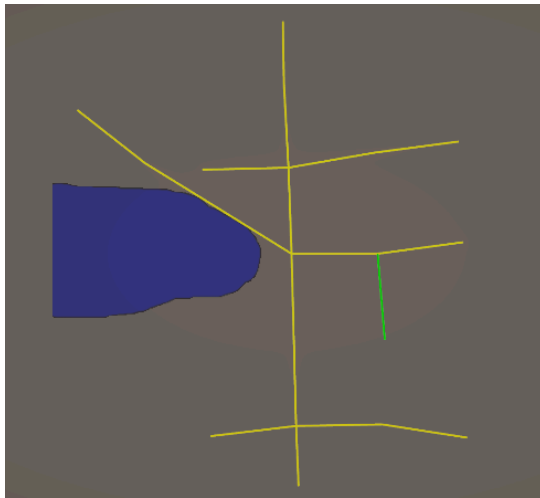
The expansion here is seeded along the major or perpendicular axis of the polygon that I find. The major axis is defined by the average direction of the longest nearly-parallel side of the polygon, with the perpendicular axis just rotated by 90 degrees from this. The minor roads are generated with no noise applied to the angle to generate more uniform grid patterns.

This concludes the main generation process, the only thing left to mention, is that this is then controlled by the generation controller allowing the user to step through or instantly generate the road map. This is not yet tied to traffic simulation, or modeling the expansion of roads over time due to usage, but this is a goal for the future. The generated roads are then converted to a mesh using a simple list of verts and tris which is rendered.

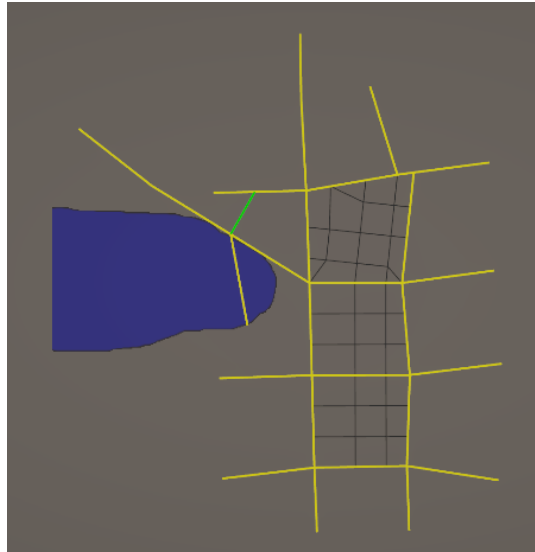
```
a = new Vector3(edge.A.Position.x, 0, edge.A.Position.y);
b = new Vector3(edge.B.Position.x, 0, edge.B.Position.y);
Vector3 dir = (b - a).normalized;
if (dir == Vector3.zero) continue;
perp = new Vector3(-dir.z, 0, dir.x) * (width * 0.5f);
}

int baseIdx = verts.Count;
verts.Add(a - perp);
verts.Add(a + perp);
verts.Add(b + perp);
verts.Add(b - perp);

tris.Add(baseIdx); tris.Add(baseIdx + 1); tris.Add(baseIdx + 2);
tris.Add(baseIdx); tris.Add(baseIdx + 2); tris.Add(baseIdx + 3);
```



Initial road segment generation



Quarter Filling

Traffic Simulation

The next main aspect of the project that I wanted to focus on was simulating basic traffic flow along the streets. This is done by spawning traffic agents along the streets in a “simulation” mode. The behaviour of the agents is fairly basic but consistent at the moment, leaving room for expansion and robustness down the line. Currently traffic agents will spawn at random vertices and have destinations at random vertices along the road map. After spawning agents will consult the pathfinding service for the path they should take to navigate. This is currently done just using Dijkstra’s since road generation is paused during simulation mode, the best routes are calculated and cached as needed since the graph is static. In the future if I wanted navigation along with graph expansion, I might look to something like A* that is used more commonly in games now and would be more robust.

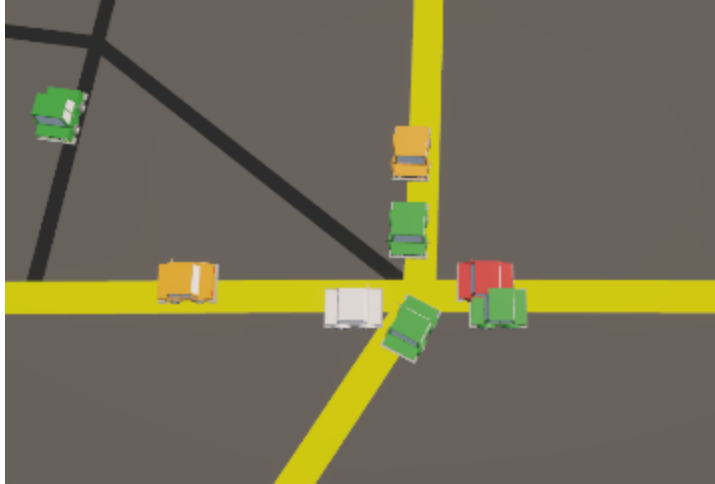
```
1 reference
public List<RoadNode>? FindPath(RoadNode start, RoadNode end)
{
    if (!_prevCache.TryGetValue(start, out var prev))
    {
        prev = RunDijkstra(start);
        _prevCache[start] = prev;
    }

    return prev.ContainsKey(end) ? Reconstruct(prev, end) : null;
}
```

The behaviour of the agents is somewhat simple, but provides decent visual representation. Agents will queue at intersections, waiting a configurable amount of time before moving through. They also know about other agents on the same road as them and will stop a given distance behind them to not run into them. Along with being offset to the side of the road, as if driving on that side, this makes the appearance actually somewhat resemble driving, though there is no randomness in the movement currently so it looks a bit more mechanical and robotic than real traffic.

```
// Release active agent after interval
if (_serving)
{
    if (_active == null || _active.State == AgentState.Finished)
    {
        _active = null;
        _serving = false;
        _timer = 0f;
    }
    else
    {
        _timer += dt;
        if (_timer >= _parameters.ReleaseInterval)
        {
            _active.ClearIntersection();
            _active = null;
            _serving = false;
            _timer = 0f;
        }
    }
}
}
```

```
float stopT = Mathf.Clamp01(1f - (_parameters.StopDistanceFromIntersection / edgeLen));
float? leadT = GetLeadingCarT(sim);
float followT = leadT.HasValue ? leadT.Value - (_parameters.FollowDistance / edgeLen) : 1f;
```



A few of the more visual features I was glad to be able to implement were basic splines to model turning through the intersection. This gave a much more gentle and curved turning motion and made the intersection navigation look much better. Additionally, adding a free 3d car model, and randomizing the colors used allowed me to get more variety in the visual element of this and make it more interesting to watch play out. Finally, I also added basic particle effects when agents reach their destination instead of just disappearing which gives some indication to the user that cars are reaching their destinations.

```
var entryPt = intersectionNode.Position
    + outDir * _parameters.StopDistanceFromIntersection
    + outRight * _parameters.LaneOffset * _parameters.LaneMultiplier;
var entryPos = new float3(entryPt.x, 0f, entryPt.y);

float dist = Vector2.Distance(WorldPosition, entryPt);
float tScale = dist * _parameters.TurnTangentScale;

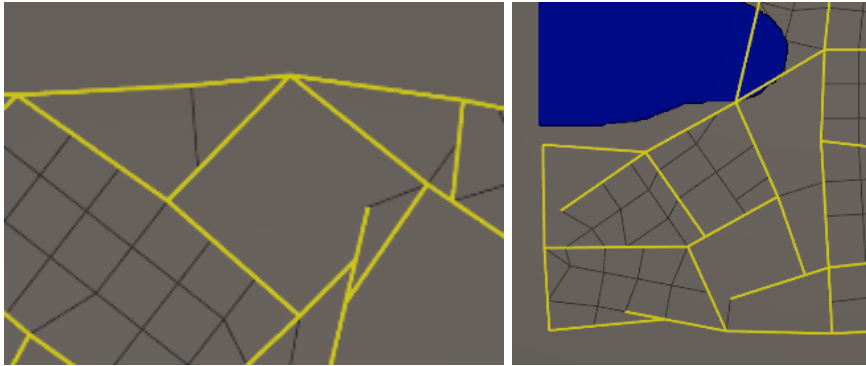
var p1 = stopPos + new float3(inDir.x, 0f, inDir.y) * tScale;
var p2 = entryPos - new float3(outDir.x, 0f, outDir.y) * tScale;

_turnCurve = new BezierCurve(stopPos, p1, p2, entryPos);
_isTurning = true;
_turnT = 0f;
```

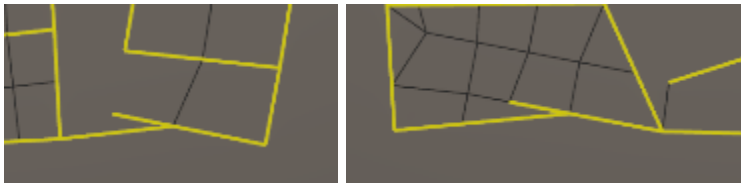
Known Bugs

There are still a number of bugs I'm working to solve, despite implementing many core features. First the quarter filling algorithm had some issues I wasn't able to resolve in the time frame. First, when snapping to a major road, minor roads don't branch further off of this. This is because of the way I set up my generation, for major nodes to generate other major nodes, but because of the design choices I made, fixing this actually required

large overhauls of my road generation that I wasn't able to finish and debug in time, so I used an older version that included this bug, but had more stable generation overall.



In addition, with regards to the road generation, there tend to be roads that get close to connecting, but don't quite and this is something I worked to fix many times. The current version is much improved over how it started, but is obviously still a bit rough, especially near the outer border of the map.



With regards to the traffic simulation, one of the main bugs that I know of is that if cars pile up far enough to fill an entire road segment, then the next car will just drive inside of the last car's position since it has no more room. To fix this, I would implement a lookahead system for the TrafficAgents, so that they won't proceed through an intersection if their next road segment is already full of cars.



Next Steps

After addressing the quarter filling bugs, there are a few directions I would like to focus on. One area is to subdivide areas enclosed by minor roads into smaller plots, and adding basic models to them. An initial plan for this would be to simply extrude the base upward to a certain height, though methods described by Parish and Müller could also be used for more interesting results (2001).

Another direction I wanted to go in, but ran short on time for, was simulating the growth of the city over time. In the methods used by Weber et al, this involved somewhat complex land-use simulation based on elevation, different building zoning, and more to estimate land value, population density and more to determine where people travel between, and which streets need expansion.

Finally, another interesting thing to explore would be adding more user controllable parameters and interaction in terms of water, elevation, and population maps, as well as other parameters that are currently only accessible internally.

Additionally to add more polish, it would be nice to improve the visuals. The roads are currently just line segments, I experimented with using splines for curves, but ran into some issues when using only 2 endpoints because of undefined tangents at the endpoints. Because of this, I made an editor toggleable spline mode, but left this off for the actual executable since there were issues. Additionally, adding textures or ground/background maps would have made it look a bit nicer overall.

Artifacts

- Source/Unity Project - <https://github.com/lukektx/CityGeneration>
- Linux Build/Executable (Tested on Linux Mint 22)-
<https://drive.google.com/file/d/1-EGxpUxSLOdOcxeGRLs5kFpzPx4C98iR/view?usp=sharing>
- Windows Build/Executable -
<https://drive.google.com/file/d/1lnJkU97s3wNpmQrzgcBCk2qDCWBUvACm/view?usp=sharing>

References

Source Papers:

Parish, Yoav I. H., and Pascal Müller. "Procedural Modeling of Cities." *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*, 2001, pp. 301–308, <https://doi.org/10.1145/383259.383292>.

Weber, Basil, et al. "Interactive Geometric Simulation of 4D Cities." *Computer Graphics Forum*, vol. 28, no. 2, Apr. 2009, pp. 481–492, <https://doi.org/10.1111/j.1467-8659.2009.01387.x>. Accessed 18 Apr. 2021.

Models:

<https://free3d.com/3d-model/cars-61713.html>